

Proyecto Final - Predicción de Cancelación de Clientes (Churn)

Introducción

El objetivo de este proyecto es desarrollar un modelo de machine learning capaz de predecir la cancelación de clientes de la empresa Interconnect. La compañía busca identificar usuarios con alta probabilidad de abandonar el servicio para implementar estrategias de retención mediante promociones y planes especiales.

Para ello, se realizó un análisis exploratorio de datos, preprocesamiento, creación de características y entrenamiento de distintos modelos de clasificación utilizando como métrica principal AUC-ROC.

Plan de Trabajo

Preguntas aclaratorias

1. ¿Existe desbalance significativo entre clientes que cancelan y clientes que permanecen activos?
 2. ¿Se espera priorizar interpretabilidad del modelo o el mejor desempeño predictivo posible?
 3. ¿Hay restricciones respecto al uso de modelos avanzados como CatBoost o LightGBM?
 4. ¿Qué variables podrían tener mayor impacto en la cancelación de clientes según el contexto de negocio?
 5. ¿Cuál sería el costo estimado para la empresa de falsos positivos y falsos negativos en la predicción de churn?
-

Plan aproximado para resolver la tarea

Paso 1: Carga y preparación de datos

Se cargarán los datasets proporcionados y se unirán mediante la columna `customerID`. Posteriormente, se revisarán tipos de datos, valores ausentes y posibles inconsistencias.

Paso 2: Análisis exploratorio de datos (EDA)

Se analizarán patrones de cancelación de clientes utilizando estadísticas descriptivas y visualizaciones para identificar variables relevantes relacionadas con churn.

Paso 3: Preprocesamiento y creación de características

Se transformarán variables categóricas, se manejarán valores faltantes y se crearán nuevas variables que puedan mejorar la capacidad predictiva del modelo.

Paso 4: Entrenamiento y evaluación de modelos

Se entrenarán distintos modelos de clasificación y se evaluarán utilizando principalmente la métrica AUC-ROC, además de exactitud como métrica complementaria.

Paso 5: Selección del modelo final y conclusiones

Se seleccionará el modelo con mejor desempeño y se presentarán conclusiones estratégicas orientadas a apoyar la retención de clientes en Interconnect.

```
In [ ]: # Librerías para la manipulación de datos
import pandas as pd
import numpy as np

# Librerías para visualización
import matplotlib.pyplot as plt
import seaborn as sns

# Configuración visual
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

# Librerías de machine learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    classification_report,
    confusion_matrix,
    roc_curve
)

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Ignorar warnings
import warnings
warnings.filterwarnings('ignore')
```

```
In [ ]: # Cargar datasets

contract = pd.read_csv('/datasets/final_provider/contract.csv')
personal = pd.read_csv('/datasets/final_provider/personal.csv')
internet = pd.read_csv('/datasets/final_provider/internet.csv')
phone = pd.read_csv('/datasets/final_provider/phone.csv')
```

```
print('contract:', contract.shape)
print('personal:', personal.shape)
print('internet:', internet.shape)
print('phone:', phone.shape)
```

contract: (7043, 8)
 personal: (7043, 5)
 internet: (5517, 8)
 phone: (6361, 2)

In []: *# Vista inicial de los datasets*

```
display(contract.head())
display(personal.head())
display(internet.head())
display(phone.head())
```

	customerID	BeginDate	EndDate	Type	PaperlessBilling	PaymentMethod	MonthlyCharges
0	7590-VHVEG	2020-01-01	No	Month-to-month	Yes	Electronic check	
1	5575-GNVDE	2017-04-01	No	One year	No	Mailed check	
2	3668-QPYBK	2019-10-01	2019-12-01 00:00:00	Month-to-month	Yes	Mailed check	
3	7795-CFOCW	2016-05-01	No	One year	No	Bank transfer (automatic)	
4	9237-HQITU	2019-09-01	2019-11-01 00:00:00	Month-to-month	Yes	Electronic check	

◀ ▶

	customerID	gender	SeniorCitizen	Partner	Dependents
0	7590-VHVEG	Female	0	Yes	No
1	5575-GNVDE	Male	0	No	No
2	3668-QPYBK	Male	0	No	No
3	7795-CFOCW	Male	0	No	No
4	9237-HQITU	Female	0	No	No

	customerID	InternetService	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupp
0	7590-VHVEG	DSL	No	Yes	No	
1	5575-GNVDE	DSL	Yes	No	Yes	
2	3668-QPYBK	DSL	Yes	Yes	No	
3	7795-CFOCW	DSL	Yes	No	Yes	
4	9237-HQITU	Fiber optic	No	No	No	



	customerID	MultipleLines
0	5575-GNVDE	No
1	3668-QPYBK	No
2	9237-HQITU	No
3	9305-CDSKC	Yes
4	1452-KIOVK	Yes

Observaciones iniciales

- Los datasets `contract` y `personal` contienen información de todos los clientes registrados.
- Los datasets `internet` y `phone` contienen menos registros, lo que indica que no todos los clientes utilizan ambos servicios.
- La columna `EndDate` sera utilizada para construir la variable objetivo relacionada con cancelacion de clientes.
- La columna `TotalCharges` probablemente requiera conversion de tipo de dato para facilitar el analisis numerico.
- Se identifican multiples variables categoricas que requeriran transformacióo para el entrenamiento de modelos de machine learning.

In []: *# Información general de Los datasets*

```
print(contract.info())
print()

print(personal.info())
print()

print(internet.info())
print()

print(phone.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerID            7043 non-null   object
1   BeginDate              7043 non-null   object
2   EndDate                7043 non-null   object
3   Type                   7043 non-null   object
4   PaperlessBilling       7043 non-null   object
5   PaymentMethod          7043 non-null   object
6   MonthlyCharges         7043 non-null   float64
7   TotalCharges           7043 non-null   object
dtypes: float64(1), object(7)
memory usage: 440.3+ KB
None
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerID            7043 non-null   object
1   gender                 7043 non-null   object
2   SeniorCitizen          7043 non-null   int64
3   Partner                7043 non-null   object
4   Dependents             7043 non-null   object
dtypes: int64(1), object(4)
memory usage: 275.2+ KB
None
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5517 entries, 0 to 5516
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerID            5517 non-null   object
1   InternetService        5517 non-null   object
2   OnlineSecurity         5517 non-null   object
3   OnlineBackup           5517 non-null   object
4   DeviceProtection       5517 non-null   object
5   TechSupport            5517 non-null   object
6   StreamingTV            5517 non-null   object
7   StreamingMovies        5517 non-null   object
dtypes: object(8)
memory usage: 344.9+ KB
None
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6361 entries, 0 to 6360
Data columns (total 2 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   customerID            6361 non-null   object
1   MultipleLines          6361 non-null   object
dtypes: object(2)
memory usage: 99.5+ KB
None
```

```
In [ ]: # Convertir TotalCharges a numérico

contract['TotalCharges'] = pd.to_numeric(
    contract['TotalCharges'],
    errors='coerce'
)

# Crear variable objetivo Churn

contract['Churn'] = contract['EndDate'].apply(
    lambda x: 0 if x == 'No' else 1
)

# Revisar valores faltantes

print(contract.isna().sum())

# Vista rápida

contract.head()
```

```
customerID      0
BeginDate       0
EndDate         0
Type            0
PaperlessBilling 0
PaymentMethod    0
MonthlyCharges  0
TotalCharges    11
Churn           0
dtype: int64
```

Out []:

	customerID	BeginDate	EndDate	Type	PaperlessBilling	PaymentMethod	Monthl
0	7590-VHVEG	2020-01-01	No	Month-to-month	Yes	Electronic check	
1	5575-GNVDE	2017-04-01	No	One year	No	Mailed check	
2	3668-QPYBK	2019-10-01	2019-12-01 00:00:00	Month-to-month	Yes	Mailed check	
3	7795-CFOCW	2016-05-01	No	One year	No	Bank transfer (automatic)	
4	9237-HQITU	2019-09-01	2019-11-01 00:00:00	Month-to-month	Yes	Electronic check	

Observaciones sobre limpieza de datos

- La columna `TotalCharges` fue convertida exitosamente a formato numerico.
- Se identificaron 11 valores faltantes en `TotalCharges` , posiblemente asociados a clientes nuevos sin historial de facturacion acumulada.

- Se creo correctamente la variable objetivo **Churn** , donde:
 - 0** representa clientes activos.
 - 1** representa clientes que cancelaron el servicio.
- La variable objetivo permitira entrenar modelos de clasificación para predecir cancelación de clientes.

In []: *# Unir datasets*

```
df = contract.merge(personal, on='customerID', how='left')

df = df.merge(internet, on='customerID', how='left')

df = df.merge(phone, on='customerID', how='left')

# Revisar dimensiones finales

print(df.shape)

df.head()
```

(7043, 21)

Out []:

	customerID	BeginDate	EndDate	Type	PaperlessBilling	PaymentMethod	Monthl
--	------------	-----------	---------	------	------------------	---------------	--------

0	7590-VHVEG	2020-01-01	No	Month-to-month	Yes	Electronic check	
1	5575-GNVDE	2017-04-01	No	One year	No	Mailed check	
2	3668-QPYBK	2019-10-01	2019-12-01 00:00:00	Month-to-month	Yes	Mailed check	
3	7795-CFOCW	2016-05-01	No	One year	No	Bank transfer (automatic)	
4	9237-HQITU	2019-09-01	2019-11-01 00:00:00	Month-to-month	Yes	Electronic check	

5 rows × 21 columns



In []: *# Valores faltantes*

```
missing_values = df.isna().sum().sort_values(ascending=False)

missing_values[missing_values > 0]
```

```
Out[ ]: StreamingMovies      1526
StreamingTV                1526
TechSupport                1526
DeviceProtection           1526
OnlineBackup               1526
OnlineSecurity             1526
InternetService            1526
MultipleLines              682
TotalCharges               11
dtype: int64
```

Observaciones sobre valores faltantes

- Los valores faltantes en variables relacionadas con servicios de internet probablemente corresponden a clientes que no contrataron este servicio.
- Los valores faltantes en `MultipleLines` pueden representar clientes sin servicio telefonico.
- Los 11 valores faltantes en `TotalCharges` probablemente pertenecen a clientes nuevos sin cargos acumulados registrados.
- Los valores faltantes identificados tienen coherencia con el contexto de negocio y no parecen originarse por errores de captura de datos.

```
In [ ]: # Reemplazar valores faltantes en servicios de internet

internet_columns = [
    'InternetService',
    'OnlineSecurity',
    'OnlineBackup',
    'DeviceProtection',
    'TechSupport',
    'StreamingTV',
    'StreamingMovies'
]

for col in internet_columns:
    df[col] = df[col].fillna('No internet service')

# Reemplazar valores faltantes en MultipleLines

df['MultipleLines'] = df['MultipleLines'].fillna('No phone service')

# Reemplazar valores faltantes en TotalCharges

df['TotalCharges'] = df['TotalCharges'].fillna(0)

# Verificación final

df.isna().sum().sum()
```

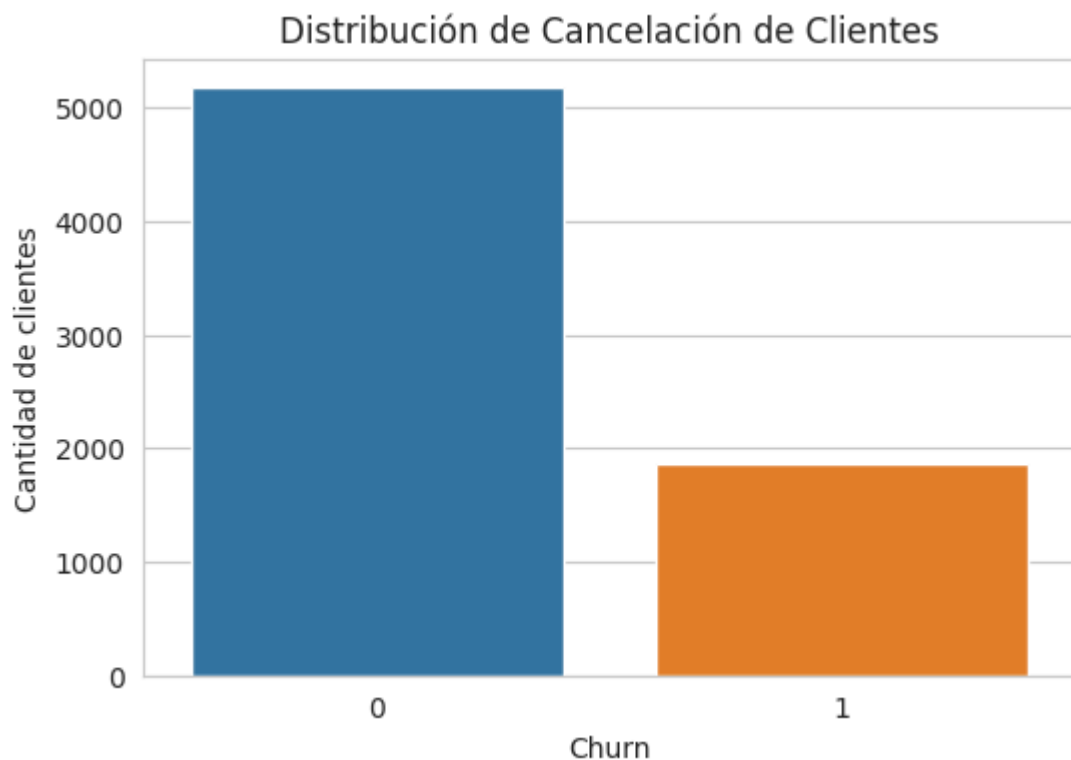
```
Out[ ]: 0
```

```
In [ ]: # Distribución de la variable objetivo

plt.figure(figsize=(6,4))
```



```
sns.countplot(  
    data=df,  
    x='Churn'  
)  
  
plt.title('Distribución de Cancelación de Clientes')  
plt.xlabel('Churn')  
plt.ylabel('Cantidad de clientes')  
  
plt.show()  
  
# Proporciones  
  
df['Churn'].value_counts(normalize=True)
```



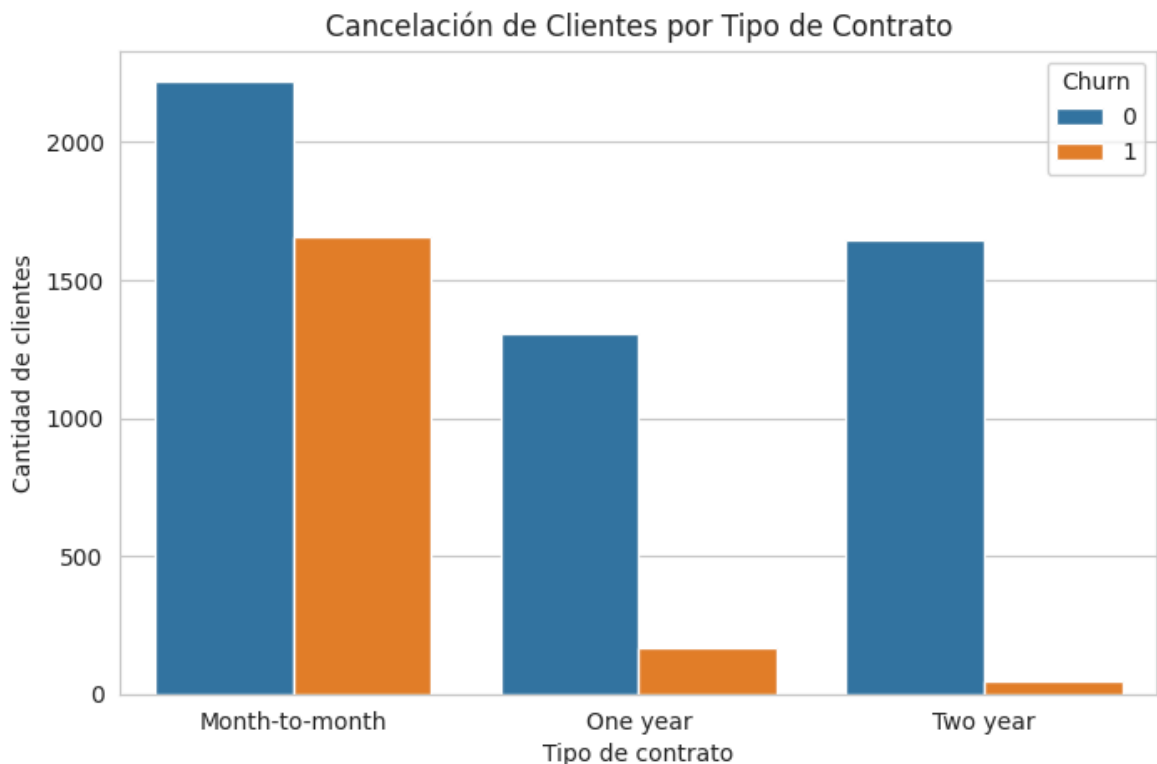
```
Out[ ]: 0    0.73463  
       1    0.26537  
       Name: Churn, dtype: float64
```

Observaciones sobre la distribución de churn

- La mayoría de los clientes permanecen activos, representando aproximadamente el 73% del total.
- Cerca del 27% de los clientes cancelaron el servicio, lo que representa una proporción significativa para el negocio.
- El dataset presenta cierto desbalance de clases, aunque no parece extremo.
- Debido a este comportamiento, la métrica AUC-ROC será especialmente importante para evaluar correctamente el desempeño de los modelos.

```
In [ ]: # Cancelación según tipo de contrato  
  
plt.figure(figsize=(8,5))
```

```
sns.countplot(  
    data=df,  
    x='Type',  
    hue='Churn'  
)  
  
plt.title('Cancelación de Clientes por Tipo de Contrato')  
plt.xlabel('Tipo de contrato')  
plt.ylabel('Cantidad de clientes')  
  
plt.show()
```



Observaciones sobre el tipo de contrato

- Los clientes con contratos mensuales (Month-to-month) presentan la mayor cantidad de cancelaciones.
- Los contratos de uno y especialmente dos años muestran niveles considerablemente menores de churn.
- Este comportamiento sugiere que contratos de mayor duración contribuyen significativamente a la retención de clientes.
- El tipo de contrato parece ser una variable altamente relevante para la predicción de cancelación.
- Desde una perspectiva de negocio, promover contratos de mayor duración podría ayudar a reducir la tasa de cancelación.

```
In [ ]: # Cancelación según tipo de servicio de internet
```

```
plt.figure(figsize=(8,5))  
  
sns.countplot(  
    data=df,
```

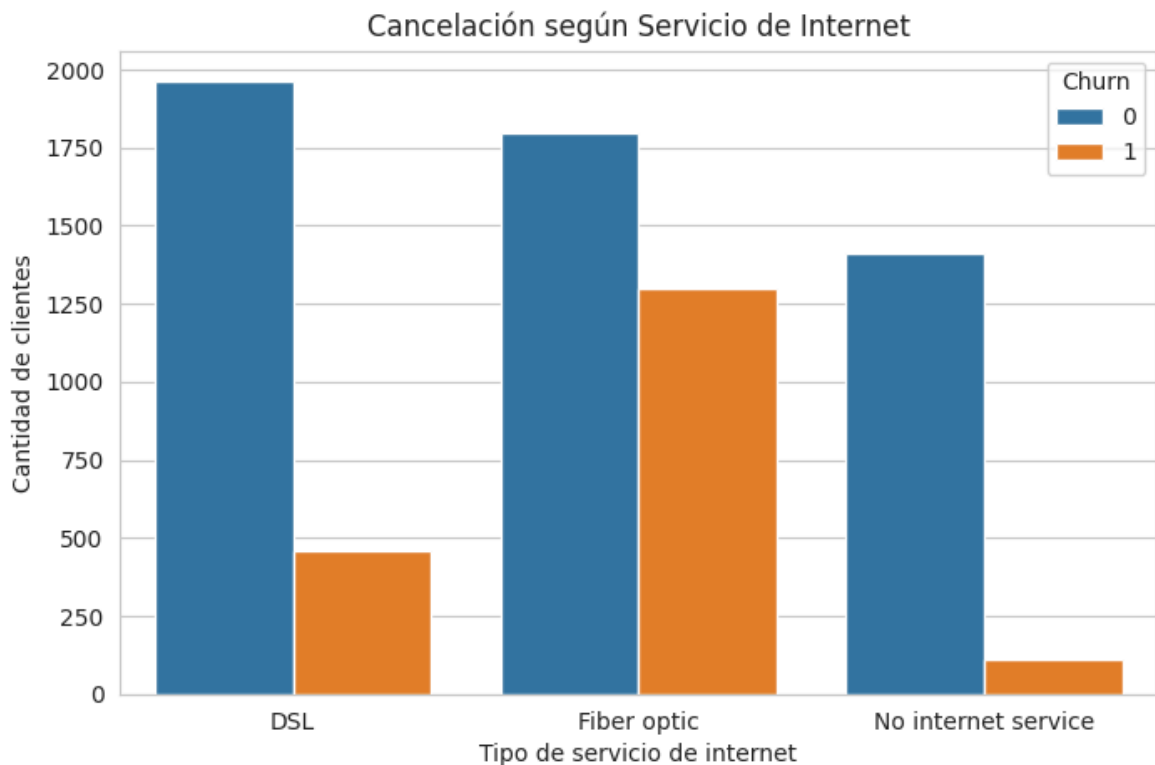
```

x='InternetService',
hue='Churn'
)

plt.title('Cancelación según Servicio de Internet')
plt.xlabel('Tipo de servicio de internet')
plt.ylabel('Cantidad de clientes')

plt.show()

```



Observaciones sobre el servicio de internet

- Los clientes con servicio **Fiber optic** presentan una proporción considerablemente mayor de cancelaciones.
- Los usuarios con servicio **DSL** muestran menor nivel de churn en comparación con fibra óptica.
- Los clientes sin servicio de internet presentan una tasa de cancelación relativamente baja.
- El tipo de servicio de internet parece ser una variable altamente influyente en el comportamiento de cancelación.
- Desde una perspectiva de negocio, podría ser útil investigar posibles causas de insatisfacción asociadas al servicio de fibra óptica.

In []: *# Distribución de cargos mensuales según churn*

```

plt.figure(figsize=(8,5))

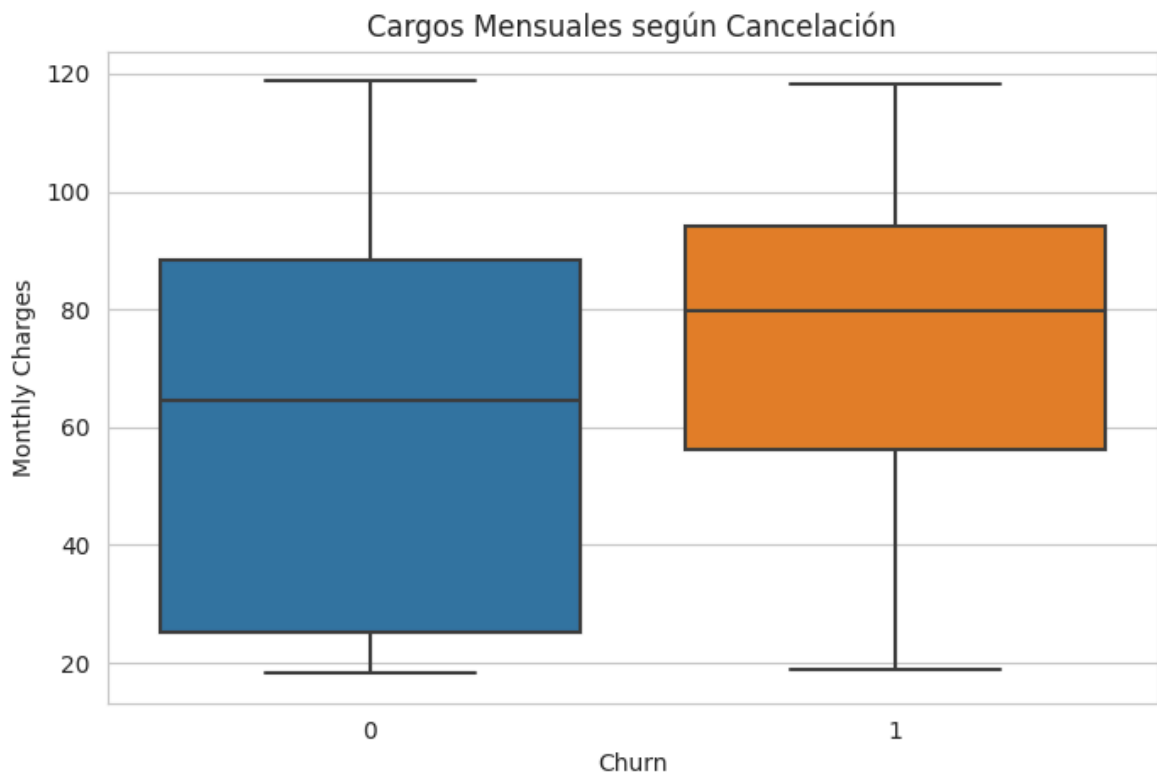
sns.boxplot(
    data=df,
    x='Churn',
    y='MonthlyCharges'
)

```

```
)

plt.title('Cargos Mensuales según Cancelación')
plt.xlabel('Churn')
plt.ylabel('Monthly Charges')

plt.show()
```



Observaciones sobre cargos mensuales

- Los clientes que cancelaron el servicio presentan, en promedio, cargos mensuales más altos.
- La mediana de `MonthlyCharges` es considerablemente mayor en clientes con churn.
- Esto sugiere que costos mensuales elevados podrían influir en la decisión de cancelación.
- Los resultados indican que `MonthlyCharges` podría ser una variable importante para los modelos predictivos.
- Desde una perspectiva de negocio, podría ser útil evaluar estrategias de retención para clientes con cargos elevados.

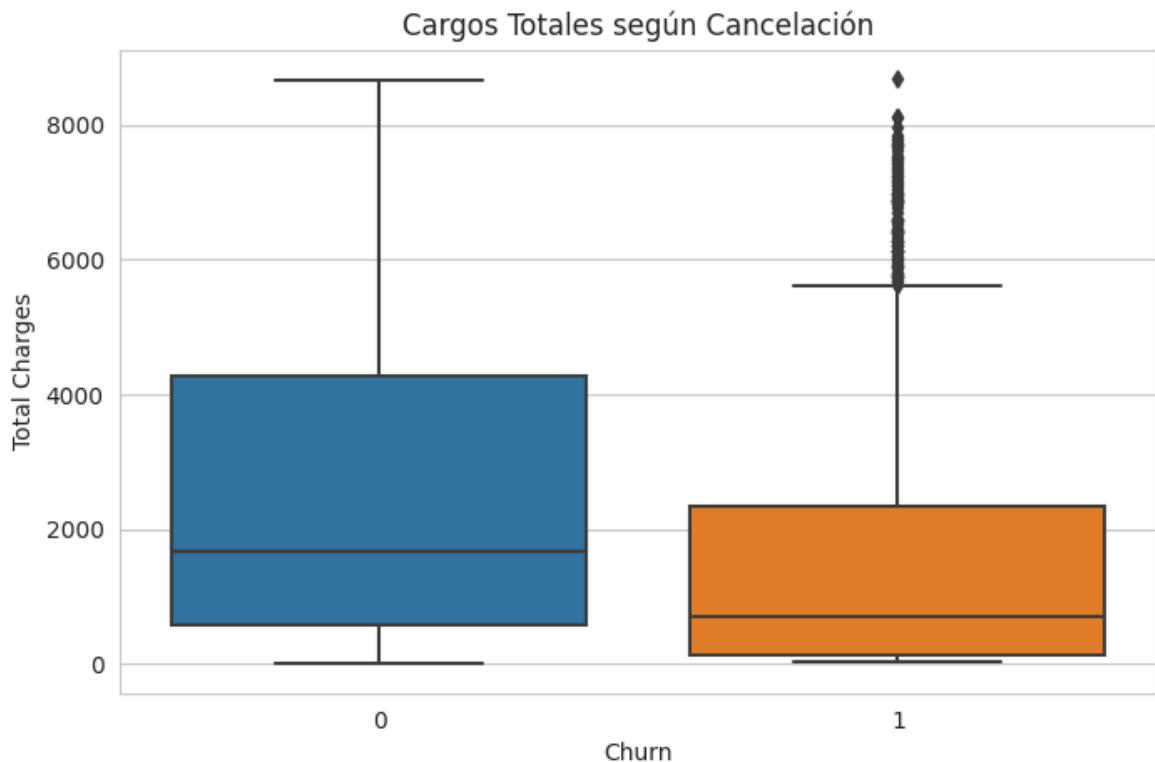
```
In [ ]: # Distribución de cargos totales según churn

plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x='Churn',
    y='TotalCharges'
)
```

```
plt.title('Cargos Totales según Cancelación')
plt.xlabel('Churn')
plt.ylabel('Total Charges')

plt.show()
```



Observaciones sobre cargos totales

- Los clientes que permanecen activos presentan cargos totales considerablemente mayores.
- Esto sugiere que los clientes con mayor antigüedad tienden a mantenerse con la empresa.
- Los clientes que cancelaron el servicio muestran menores valores acumulados de `TotalCharges`, indicando posibles cancelaciones tempranas.
- Se identifican algunos valores atípicos en clientes con churn, posiblemente asociados a usuarios con servicios de alto costo o larga permanencia previa.
- La variable `TotalCharges` parece aportar información relevante sobre fidelización y permanencia de clientes.

```
In [ ]: # Convertir fechas a formato datetime

df['BeginDate'] = pd.to_datetime(df['BeginDate'])

# Fecha de referencia del dataset

reference_date = pd.to_datetime('2020-02-01')

# Crear variable tenure en meses aproximados

df['tenure_months'] = (
    (reference_date - df['BeginDate']).dt.days / 30
```

```

).astype(int)

# Vista rápida

df[['BeginDate', 'tenure_months']].head()

```

```

Out[ ]:

```

	BeginDate	tenure_months
0	2020-01-01	1
1	2017-04-01	34
2	2019-10-01	4
3	2016-05-01	45
4	2019-09-01	5

```

In [ ]: # Antigüedad según cancelación

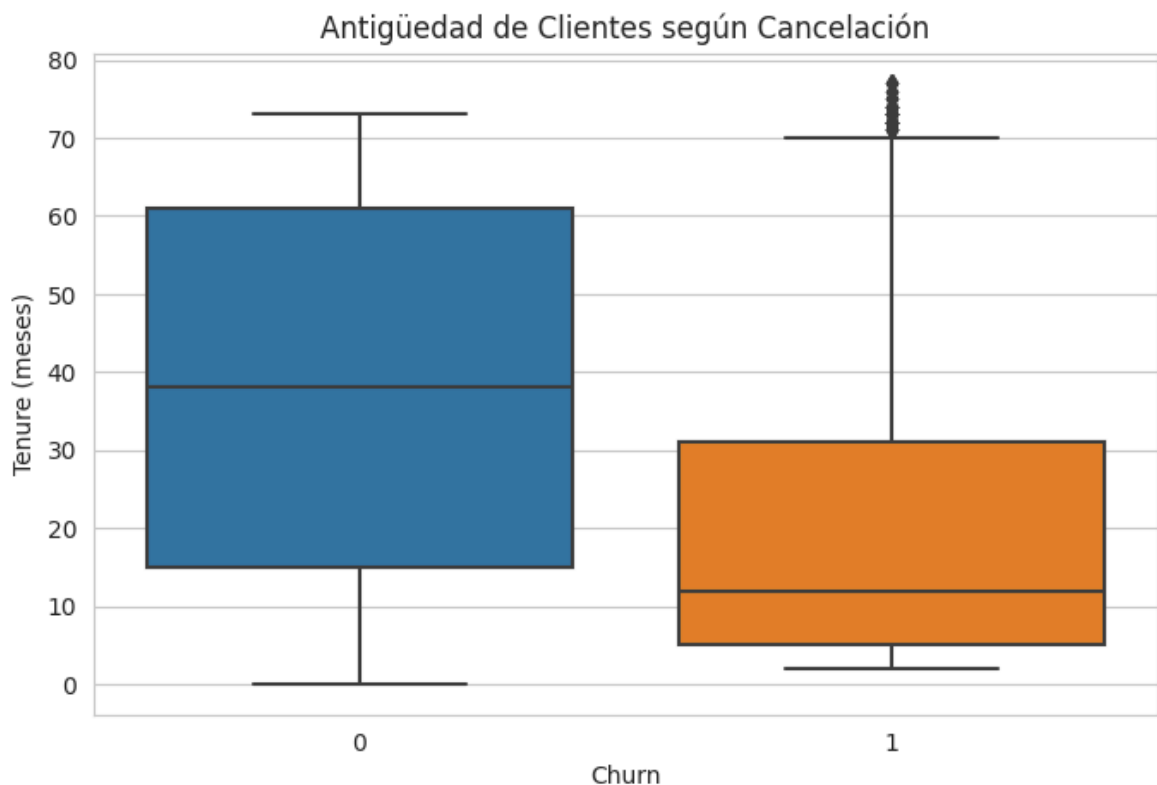
plt.figure(figsize=(8,5))

sns.boxplot(
    data=df,
    x='Churn',
    y='tenure_months'
)

plt.title('Antigüedad de Clientes según Cancelación')
plt.xlabel('Churn')
plt.ylabel('Tenure (meses)')

plt.show()

```



Observaciones sobre antigüedad de clientes

- Los clientes que permanecen activos presentan una antigüedad considerablemente mayor.
- Los usuarios que cancelaron el servicio tienden a tener pocos meses de permanencia.
- Esto sugiere que el riesgo de cancelación es más alto durante las primeras etapas de relación con la empresa.
- La variable `tenure_months` parece ser una de las características mas influyentes para predecir churn.
- Desde una perspectiva de negocio, implementar estrategias de retencion temprana podría ayudar a reducir cancelaciones.

In []: *# Eliminar columnas que no utilizaremos*

```
df_model = df.drop(
    columns=[
        'customerID',
        'BeginDate',
        'EndDate'
    ]
)

# Variables categóricas

categorical_columns = df_model.select_dtypes(
    include='object'
).columns

# Aplicar One-Hot Encoding

df_encoded = pd.get_dummies(
    df_model,
    columns=categorical_columns,
    drop_first=True
)

# Dimensiones finales

print(df_encoded.shape)

df_encoded.head()
```

(7043, 30)

Out[]:

	MonthlyCharges	TotalCharges	Churn	SeniorCitizen	tenure_months	Type_One year	Type
0	29.85	29.85	0	0	1	0	
1	56.95	1889.50	0	0	34	1	
2	53.85	108.15	1	0	4	0	
3	42.30	1840.75	0	0	45	1	
4	70.70	151.65	1	0	5	0	

5 rows × 30 columns



In []:

```
# Separar variables y target

X = df_encoded.drop('Churn', axis=1)
y = df_encoded['Churn']

# Dividir datos

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Ver dimensiones

print('X_train:', X_train.shape)
print('X_test:', X_test.shape)

print('y_train:', y_train.shape)
print('y_test:', y_test.shape)
```

X_train: (5282, 29)

X_test: (1761, 29)

y_train: (5282,)

y_test: (1761,)

In []:

```
# Modelo Logistic Regression

log_model = LogisticRegression(max_iter=1000)

# Entrenamiento

log_model.fit(X_train, y_train)

# Predicciones

y_pred_log = log_model.predict(X_test)

# Probabilidades para AUC-ROC

y_prob_log = log_model.predict_proba(X_test)[:, 1]
```



```
# Métricas

log_accuracy = accuracy_score(y_test, y_pred_log)
log_auc = roc_auc_score(y_test, y_prob_log)

print('Accuracy:', log_accuracy)
print('AUC-ROC:', log_auc)
```

Accuracy: 0.7950028392958546

AUC-ROC: 0.8359418697397643

Resultados - Logistic Regression

- El modelo de regresión logística alcanzó un AUC-ROC aproximado de 0.836.
- El desempeño obtenido indica buena capacidad para distinguir entre clientes que cancelan y clientes que permanecen activos.
- La exactitud alcanzada fue cercana al 79.5%.
- Los resultados sugieren que las variables seleccionadas contienen información relevante para la predicción de churn.
- Este modelo servirá como baseline para comparar modelos más complejos posteriormente.

```
In [ ]: # Modelo Decision Tree

tree_model = DecisionTreeClassifier(
    random_state=42,
    max_depth=5
)

# Entrenamiento

tree_model.fit(X_train, y_train)

# Predicciones

y_pred_tree = tree_model.predict(X_test)

# Probabilidades

y_prob_tree = tree_model.predict_proba(X_test)[:, 1]

# Métricas

tree_accuracy = accuracy_score(y_test, y_pred_tree)
tree_auc = roc_auc_score(y_test, y_prob_tree)

print('Accuracy:', tree_accuracy)
print('AUC-ROC:', tree_auc)
```

Accuracy: 0.8154457694491766

AUC-ROC: 0.8666220970448355

Resultados - Decision Tree

- El modelo Decision Tree alcanzo un AUC-ROC aproximado de 0.867, superando claramente el desempeño de Logistic Regression.
- La exactitud obtenida fue cercana al 81.5%.
- Los resultados sugieren que existen relaciones no lineales importantes entre las variables y la cancelación de clientes.
- El arbol de decision parece capturar patrones complejos asociados al churn de manera mas efectiva.
- El desempeño alcanzado posiciona este modelo como un candidato solido para la solucion final.

```
In [ ]: # Modelo Random Forest

forest_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

# Entrenamiento

forest_model.fit(X_train, y_train)

# Predicciones

y_pred_forest = forest_model.predict(X_test)

# Probabilidades

y_prob_forest = forest_model.predict_proba(X_test)[:, 1]

# Métricas

forest_accuracy = accuracy_score(y_test, y_pred_forest)
forest_auc = roc_auc_score(y_test, y_prob_forest)

print('Accuracy:', forest_accuracy)
print('AUC-ROC:', forest_auc)
```

Accuracy: 0.8228279386712095

AUC-ROC: 0.8715219974251115

Resultados - Random Forest

- El modelo Random Forest obtuvo el mejor desempeño entre los modelos evaluados.
- Se alcanzo un AUC-ROC aproximado de 0.872 y una exactitud cercana al 82.3%.
- El modelo mostro mejor capacidad de generalización en comparación con Decision Tree individual.
- Los resultados indican que Random Forest captura adecuadamente relaciones complejas entre variables y churn.
- Este modelo se posiciona como el candidato principal para la solución final del problema de negocio.

In []: *# Comparación de modelos*

```
results = pd.DataFrame({
    'Modelo': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest'
    ],
    'Accuracy': [
        log_accuracy,
        tree_accuracy,
        forest_accuracy
    ],
    'AUC-ROC': [
        log_auc,
        tree_auc,
        forest_auc
    ]
})

results.sort_values(
    by='AUC-ROC',
    ascending=False
)
```

Out []:

	Modelo	Accuracy	AUC-ROC
2	Random Forest	0.822828	0.871522
1	Decision Tree	0.815446	0.866622
0	Logistic Regression	0.795003	0.835942

Conclusiones Ejecutivas

Resumen Ejecutivo

- Se analizaron 7,043 clientes de Interconnect.
- La tasa de cancelación observada fue aproximadamente del 26.5%.
- Los contratos mensuales presentaron el mayor riesgo de churn.
- Los clientes con menor antigüedad mostraron mayor probabilidad de cancelar el servicio.
- Random Forest fue el modelo con mejor desempeño.
- El modelo final alcanzó un AUC-ROC de 0.872 y una exactitud cercana al 82.3%.

Resultados principales

- El análisis exploratorio permitió identificar variables altamente relacionadas con la cancelación de clientes, especialmente:
 - tipo de contrato,
 - servicio de internet,

- cargos mensuales,
 - cargos totales,
 - y antigüedad de clientes.
- Los clientes con contratos mensuales y servicio de fibra óptica presentaron mayores niveles de churn.
 - Los usuarios con menor antigüedad mostraron una tendencia significativamente mayor a cancelar el servicio.
 - Los cargos mensuales elevados también parecen estar asociados con mayor riesgo de cancelación.
-

Evaluación de modelos

Se entrenaron y evaluaron distintos modelos de clasificación utilizando AUC-ROC como métrica principal.

El modelo con mejor desempeño fue Random Forest, alcanzando:

- Accuracy aproximada de 82.3%
- AUC-ROC aproximado de 0.872

Los resultados obtenidos indican que el modelo tiene buena capacidad predictiva para identificar clientes con riesgo de cancelación.

Recomendaciones de negocio

- Promover contratos de mayor duración podría ayudar a reducir la tasa de cancelación.
 - Implementar estrategias de retención temprana para clientes nuevos puede disminuir churn en las primeras etapas.
 - Sería recomendable investigar posibles causas de insatisfacción en clientes con servicio de fibra óptica.
 - Clientes con cargos mensuales elevados podrían beneficiarse de promociones o planes personalizados para mejorar retención.
-

Conclusión general

El proyecto permitió desarrollar un modelo predictivo capaz de identificar clientes con mayor probabilidad de cancelar el servicio. Los resultados obtenidos pueden servir como apoyo para estrategias de retención y toma de decisiones basadas en datos dentro de Interconnect.

In []: